

Markdown analyzer

Application for analyzing STePSEnHECs-PaCt markdown

Students: Bc. Trong Quoc Huy Dinh, Bc. Břetislav Milota,
Bc. Martin Reich, Bc. Eliška Toncarová
Student IDs: A25N0040P, A25N0052P, A25N0056P, A25N0061P
Emails: qhdt@students.zcu.cz, bmilota@students.zcu.cz, reichm@students.zcu.cz,
eliska@toncar.cz
Date: May 16, 2026

Contents

1	Introduction	2
2	Technologies Used	2
3	Application Architecture	2
3.1	Diagrams	2
3.2	Modules	3
3.2.1	Core	3
3.2.2	Ui	5
3.2.3	Utils	6
4	Markdown File Validation	7
4.1	Validation Principles	8
4.2	Validation Rules Overview	8
5	User Interface	9
5.1	Main Window Layout	10
5.2	Markdown Scene	10
5.3	Git Scene	10
6	Testing	10
6.1	Unit Tests	10
6.2	Test Scenarios	12
7	Future Extensions	12
8	Build Guide	13
8.1	Requirements	13
8.2	End User: Running the Program	13
8.3	Building the Executable (Build Scripts)	14
8.4	Developer Environment and Dependencies	14
8.5	Notes	14
A	Appendices	16
A.1	Diagrams	17
A.2	Test Scenarios	22

1 Introduction

Markdown analyzer is a desktop application designed for analyzing and editing markdown files according to the STePSEnHECs-PaCt pattern catalog. The application also features a Git client for easier saving of changes made to markdown files. The application is executable on Windows and macOS operating systems.

2 Technologies Used

This section provides an overview of the main technologies, libraries, and tools used during the design and development of the application.

Programming language:

- **Python 3.10**

Libraries used:

- **PyQt6:** A robust framework for creating cross-platform desktop interfaces.
- **Python-Markdown (markdown):** A standard library for parsing source text and converting it to HTML for Live Preview purposes.
- **PyMdown Extensions (pymdownx):** A set of extensions for the Markdown parser that provides the application with support for modern syntax.
- **Git:** Native support for version control in local repositories.

3 Application Architecture

This section describes the application's architecture, its division into main modules, and data flows during the processing of markdown files or communication with the Git version control system.

3.1 Diagrams

The following subsections summarize the dependencies between modules and the data flow during key application activities. For the sake of clarity, the diagrams themselves have been moved to **Appendix A.1**. The display and analysis diagrams also include the editor and analyzer layers to show the context of working with files across the entire Markdown view.

Module Dependency Diagram

Diagram 1 (see Appendix A.1) illustrates how individual application classes are organized into layers and the direction of imports. The entry point is `main.py`, which initializes the central `Application` class. This class holds references to all managers (e.g., `FileManager`) and the `MainWindow`. The managers, via UI components (`MarkdownScene`, `TabWidget`, etc.), call the data layer – `MarkdownAnalyzer` with its validators, parsers (`MarkdownParser`, `TemplateParser` sharing `BaseParser`), `FileHelper`, `FileMatcher`, and the `utils.git` package. The arrow $A \rightarrow B$ means that module A imports (depends on) module B; colors distinguish the entry point, managers, UI modules, and data modules.

Data Flow During Git Operations

Diagrams 2 and 3 (see Appendix A.1) describe two parallel flows. **Flow A** (diagram 2) represents user-triggered Git operations (*status*, *fetch*, *pull*, *push*, *export staged*): a click in the `GitScene` or menu passes through `GitManager.start_operation`, which creates a `GitOperationWorker` (`QThread`) and runs the corresponding function from the `utils.git` package over the `GitPython` library within it. The result (`GitResult`) is returned to the GUI thread via `operation_output` and `operation_finished` signals and is printed in the `GitScene` console.

Flow B (diagram 3) shows automatic staging after saving a file: `FileManager` invokes `MarkdownAutoStager.stage_file_delayed`, which uses a single-shot `QTimer` (1 s) to trigger the synchronous `stage_markdown_files` and reports the result via `file_staged` or `staging_failed` signals.

Data Flow During Markdown File Display

Diagram 4 (see Appendix A.1) shows the path from opening a file (toolbar, double-click in the explorer, session restore, or clicking a link in the preview) to the two concurrent views. `FileManager.load_markdown_file` delegates validation and byte loading to `FileHelper` (with encoding detection) and passes the content to `TabManager.load_file_into_tab`. From there, data flows in two directions: to the `MarkdownEditorWidget` (`MarkdownHighlighter` performs block-by-block syntax highlighting and renders the *editor pane*) and – via the signal `textChanged` and `EditorManager.update_live_preview` – to the markdown library with `pymdownx` extensions, whose HTML output is wrapped in default CSS and displayed in a `QTextBrowser` as the *preview pane*.

Data Flow During Analysis and Validation

Diagram 5 (see Appendix A.1) illustrates the validation process triggered after loading or saving a markdown file. `FileManager._analyze_current_file` parallelly retrieves two different things: a `ParsedDocument` from `MarkdownParser.parse_content` and the `TemplateRules` for the correct template, which `FileMatcher` locates based on the file's parent directory. Subsequently, both pieces of data flow into `MarkdownAnalyzer.validate_structure`, which runs four groups of checks (`FormattingValidator`, `ContentValidator`, custom H1 and section checks, and `LinkValidator`). The resulting analysis dictionary (`warnings` + `passed`) is converted into text by the `generate_report` method, and `TabManager.set_analysis` displays it line-by-line in the `analyzer_list` panel as color-coded `QListWidgetItem`s below the editor and preview.

3.2 Modules

This section describes the logical and directory structure of the application's source code. The project architecture is designed modularly, with a clear separation of the user interface (presentation layer) from the application logic and auxiliary tools.

3.2.1 Core

The `core` module constitutes the actual heart of the entire application. All business logic, state management, component linking, and analytical tools are concentrated here.

- **analyzer** – This package contains all the logic for structural, content, and formatting checks of markdown files against STePSEnHECs-PaCt template rules. The input to the analyzer is a pair consisting of a `ParsedDocument` (the analyzed file) and `TemplateRules` (the corresponding template); the output is a list of warnings (`warnings`) and successfully passed checks (`passed`), which the `TabManager` projects into the Analyzer Output panel.

- **MarkdownAnalyzer** (`core/analyzer/markdown_analyzer.py`):

The main orchestrator of the analysis. It runs the validators (`FormattingValidator`, `ContentValidator`, `LinkValidator`) and performs document-level checks itself: the number of H1 headings, H1 match with the filename, H1 prefix match with the template, presence and order of required sections, detection of sections unknown to the template, and checking the correct heading levels. It also validates the document's breadcrumb line (number of parts, fixed values, existence of files referenced from breadcrumbs). For special documents (`template-publication`, in-memory templates for *facets* and *publications* indexes), it skips selected checks. It can convert the results into a text report using the `generate_report()` method.

- **ContentValidator** (`core/analyzer/content_validator.py`):

Checks the content under individual headings. It verifies that required sections are not empty, that the set of found content types (`found_types`) is a superset of the types expected by the template (`expected_types`), that table headers match the template, that bullet lists contain the required prefixes (e.g., (+), (-)), and that a table exists where the template requires it. For grouped sections (A–Z blocks), it compares the set of actual letters present with the one defined by the template. It also detects italicized instructions taken from the template (e.g., *remove or replace this text*), which have no place in the final document.

- **FormattingValidator** (`core/analyzer/formatting_validator.py`):

Focuses on low-level text formatting checks. It monitors odd numbers of asterisks (unclosed bold `**` or italic `*`, bullet markers are filtered out from the line), images without alt text (``), correct table composition, and the presence of the Unicode *replacement* character (`\ufffd`), which indicates corrupted file encoding.

- **LinkValidator** (`core/analyzer/links_validator.py`):

Validates markdown links project-wide using a global file index (`project_index`). It verifies that every relative link to an `.md` file exists within the project, that cross-references between patterns in the *Related Patterns* section are reciprocal (if pattern A links to pattern B, B must link back to A), and that the link label matches the target file's name or one of its aliases listed in the *Also Known As* section. For index templates, the alias check can be skipped using the `skip_alias_check` flag.

- **managers** – This package serves as the main controller layer of the application. Classes in this folder separate application logic from the user interface (UI) itself. Their task is to coordinate processes, handle UI events, and manage communication between visual components and low-level tools (such as the analyzer, Git, or file system).

- **EditorManager** (`core/managers/editor_manager.py`):

Responsible for managing the text editor itself and its auxiliary visual panels. It handles state changes of the toggles for displaying the live preview and analyzer

output. Its key function is dynamically converting raw markdown into formatted HTML code using the `markdown` library and a broad set of `pymdownx` extensions. It takes the content from the current tab (`TabManager`) and ensures its immediate visual representation in the `Live Preview` panel.

– **ErrorManager** (`core/managers/error_manager.py`):

Centralizes the display of pop-up dialogs and error messages across the entire application. Using class (static) methods, it provides a unified interface for invoking informational messages, warnings, critical errors, or interactive queries (such as choosing to save/discard changes when closing the application). It internally implements all these alerts via the standard `QMessageBox` UI component.

– **FileManager** (`core/managers/file_manager.py`):

Coordinates document operations in the user interface: loading templates from the directory, opening and saving markdown files in cooperation with `TabManager`, triggering analysis via `MarkdownAnalyzer`, and handling the file explorer. For reading, writing, and dialogs, it delegates to `FileHelper`. Connections with the window, menu, and tabs are established from the `Application` class, including handling clicks on links in the preview.

– **GitManager** (`core/managers/git_manager.py`):

Serves as the manager for operations with the Git version control system connected to the user interface. To prevent the application from freezing during network or disk operations, it delegates the execution of commands (e.g., `push`, `pull`, `commit`) to an asynchronous worker thread, `GitOperationWorker`. It then informs the UI about results, progress, and errors using an event (signal) system.

– **TabManager** (`core/managers/tab_manager.py`):

Manages the lifecycle of tabs in the main workspace (`MarkdownScene`). Using the helper data class `TabState`, it maintains context about each open tab, including the path to the loaded file and the indication of unsaved changes (the *dirty state*, represented by an asterisk in the title). It handles the logic of opening, closing, and switching tabs, passes text changes for further processing, and writes the formatted analyzer results into the corresponding UI panel in the specific tab.

- **application.py** – The `Application` class represents the central hub of the entire application. It serves to initialize all key components (main window, analyzers, managers, helper tools) and to connect the user interface with the application logic. It provides the signal wiring—connecting UI events (such as button clicks, menu selections, or checkbox changes) with specific actions in the individual managers. Furthermore, it controls the application lifecycle, which includes applying the graphical theme (QSS), starting the main event loop, safely loading the previous state at startup (e.g., window position, open tabs, last directory), and saving it upon application exit.

3.2.2 Ui

The `ui` module contains the definition of the application’s user interface:

- **assets** – application assets (icons and QSS themes).

- **components** – ui components used in individual scenes.
- **scenes** – application scenes.
- **main_window.py** – main application window definition.

3.2.3 Utils

The `utils` module contains the application’s helper tools.

- **FileHelper** (`utils/file_helper.py`):

The `FileHelper` class concentrates low-level file operations and PyQt6 system dialogs. It provides the selection of one or more markdown files and a working directory (`QFileDialog`), validates file existence, type, and read permissions, handles reading and writing with encoding detection (UTF-8, optionally the `charset_normalizer` library, fallback CP1252), and remembers the encoding for consistent saving. It also searches for markdown files in a directory (with recursion options), guards against unreasonable content size according to `FileConstants`, and assists in resolving relative links from the preview (`resolve_relative_markdown_link`).

- **Git Operations** (`GitManager`, `utils/git` package):

Management of Git commands is handled by (`core/managers/git_manager.py`), which runs `GitOperationWorker` (derived from `QThread`) in its own thread. Only a maximum of one operation can run at a time; further requests are rejected with a message to the user. Depending on the operation type, the worker calls functions from the `utils/git` package (repository status, fetch, pull, push including commit message, staging, exporting staged changes to an archive, etc.), while repository interaction occurs via the `GitPython` library. Results are passed back as a `GitResult`. The dangerous export action requires confirmation via a dialog regarding potential data loss. Manager signals update the Git Scene (console, enabling controls); the specific wiring is in `Application`.

- **Parsers** (`utils/parsers` package):

The `utils/parsers` package converts the raw text of markdown files and templates into structured data classes, which the analyzer subsequently works with. To avoid repeating heading, breadcrumbs, and content type recognition logic across two modules, all regex and classification logic is centralized into the shared `base_parser` module.

- **base_parser** (`utils/parsers/base_parser.py`):

A module of shared parsing primitives. It contains functions `parse_heading_line()` (parses a heading line into level, plain text, link, and for templates, *optional* and italicized *variable* placeholder indicators), `split_h1()` (splits an H1 into a prefix and a value based on a colon), `parse_breadcrumbs()` (distinguishes a breadcrumb line from template italic instructions), and `classify_content_line()` (classifies a content line into types: text, bullet list, table, image, link, horizontal rule, or footnote, and extracts table headers and bullet prefixes). All other parsers operate precisely on these functions.

- **MarkdownParser** (`utils/parsers/markdown_parser.py`):

Converts a loaded markdown file into a `ParsedDocument`. The output is a pair:

DocumentMeta (file path, breadcrumbs in original and normalized form, H1 prefix, and H1 value) and a list of HeadingInfo (heading level, text, line number, and nested ContentInfo with found content types, bullet prefixes, table headers, and *raw_lines* for later formatting checks). Continuous runs of A–Z headings are automatically collapsed into a single `is_group=True` record so they can be compared to the template as a whole. Furthermore, the `get_link_metadata()` method extracts aliases (*Also Known As* section) and cross-references (*Related Patterns*) from the document; the results are projected into the global `project_index`, which the LinkValidator operates on.

– **TemplateParser** (`utils/parsers/template_parser.py`):

The TemplateParser class represents the MarkdownParser analogy for template files. The output is TemplateRules containing DocumentRules (expected breadcrumbs, H1 prefix) and a list of HeadingRules with nested ContentRules (expected content types, required bullet prefixes, table headers). The parser additionally detects (*optional*) markers and italicized *variable* placeholders in headings, and, just like the document parser, collapses A–Z blocks. Loaded templates are maintained in an internal `templates` map and looked up by name.

– **FileMatcher** (`utils/file_matcher.py`):

The FileMatcher class pairs a markdown file with the corresponding template based on the parent directory’s name (e.g., files in the `patterns/` directory are validated against the `template-pattern` template). Special cases like *facets* indexes and publication files are handled using in-memory templates from the `facets_index_template` module.

– **facets_index_template** (`utils/facets_index_template.py`):

Defines TemplateRules programmatically, without loading from a file, for documents that do not have a corresponding `.md` template on disk (indexes by facet, list of publications). These templates have looser rules—they allow variable H1s and arbitrary text content.

– **Constants and Exceptions** (`utils/constants.py`, `utils/exceptions.py`):

The `constants.py` file centralizes all “magic values”—regular expressions for headings, links, and breadcrumbs, allowed file extensions, size limits, icons for the report (ICON_OK, ICON_WARNING), and constants for Git operations. `exceptions.py` file defines a hierarchy of custom exceptions (BaseAppException, GitException, FileNotFoundError, InvalidInputError, ...) with support for human-readable messages and dialog titles, allowing the ErrorManager to display consistent error messages.

4 Markdown File Validation

Validation of markdown files is the application’s core feature. The goal is to verify that every document in the STePSEnHECs-PaCt catalog complies with the rules of the respective template—it has correctly structured headings, breadcrumbs, sections in the expected order, meaningful content under each section, and functional cross-references between patterns. The data flow during validation is shown in figure 5.

4.1 Validation Principles

The validation process is divided into three steps:

1. **Parsing the document and template.** `MarkdownParser` converts the markdown file content into a `ParsedDocument` (metadata, list of headings, and structured content under each). `TemplateParser` converts the corresponding template into `TemplateRules`. The matching of a document with a template is handled by `FileMatcher` based on the parent directory name (e.g., `patterns/` → `template-pattern`); for index files, in-memory templates from the `facets.index.template` module are used.
2. **Running partial validators.** `MarkdownAnalyzer`, acting as the orchestrator, sequentially runs the `FormattingValidator` (text formatting), `ContentValidator` (content of individual sections), structural and breadcrumb checks within its own body, and finally the `LinkValidator` (cross-references within the entire project, provided a global `project_index` is available). Each validator operates on shared data classes and returns two lists: `warnings` (found errors) and `passed` (successfully fulfilled checks).
3. **Aggregation and display of results.** The results of all validators are merged, sorted by line number, and passed to the `TabManager`, which displays them in the Analyzer Output panel with color-coding for successes, warnings, and errors.

4.2 Validation Rules Overview

The following table summarizes all checked rules. The *Where implemented* column refers to the check's location in the code: `document_loader` = during parsing in `MarkdownParser`, *validator* = one of the validators in the `core/analyzer` package, *both* = the rule is partially detected during parsing (storing necessary data in `ParsedDocument`) and finally evaluated by a validator.

#	Category	What is verified	Where implemented
1	Document structure	The document must have exactly one H1 heading.	<code>document_loader</code>
2	Document structure	<code>dokument.h1_value</code> (case-insensitive) == file stem.	<code>document_loader</code>
3	Document structure	The H1 prefix must match what the template defines.	both
4	Document structure	All non-optional headings from the template must exist in the document.	both
5	Document structure	If an optional section exists, its text must match the template.	both
6	Document structure	Sections must be in the same order as in the template.	both
7	Document structure	The document does not contain headings unknown to the template.	both
8	Document structure	Sections must have the correct level (<code>#</code> , <code>##</code> , <code>###...</code>) according to the template.	both
9	Breadcrumbs	The document must have a breadcrumb line.	<code>document_loader</code>

#	Category	What is verified	Where implemented
10	Breadcrumbs	The number of parts separated by 'ı' must match the template.	both
11	Breadcrumbs	Fixed parts (not the last one) must textually match the template.	both
12	Breadcrumbs	Every [label](url) in breadcrumbs points to an existing file.	validator
13	Section content	Non-optional sections must have at least some content.	both
14	Section content	found_types must be a superset of expected_types (excluding 'text').	both
15	Section content	Table columns must match the template.	both
16	Section content	If the template defines bullet_prefixes, the document must contain them.	both
17	Section content	Italic instructions from the template (e.g., 'remove or replace. . .') must not remain in the document.	both
18	Section content	The document must have a complete set of letters as per the template.	both
19	Section content	A table exists, but can be empty.	both
20	Formatting	Odd number of ** on a line = unclosed bold.	both
21	Formatting	Odd number of * (outside a bullet marker) = unclosed italic.	both
22	Formatting	! [] (url) without alt text = accessibility error.	both
23	Formatting	Presence of the replacement character (□, \ufffd) = corrupted file encoding.	both
24	Formatting	Correctly formatted table.	both
25	Links	[] (url) or [label]() = badly formatted link.	document_loader
26	Links	[label](relative_path) must point to an existing file.	document_loader
27	Links	If pattern A links to pattern B in the 'Related Patterns' section, then B must have a link back to A.	validator
28	Links	If there is a link to a pattern under a different name (alias) in an A-Z section, verify the target file exists and the target pattern has the alias in the 'Also Known As' section.	validator

Table 1: Overview of validation rules applied to markdown files.

5 User Interface

The application fully utilizes its own Dark Mode theme and vector icons. The interface is divided into two main working views (scenes), which you can toggle between.

5.1 Main Window Layout

- **Sidebar:** A narrow vertical panel on the left side of the window, used for quick switching between the Markdown editor and the Git management interface.
- **Top Menu and Toolbar:** Provides access to the most frequently used actions (opening files and folders, saving, running Git operations) and contains keyboard shortcuts.

5.2 Markdown Scene

This view is the primary interface for editing and analyzing markdown files.

- **File Explorer:** A left side panel showing the tree structure of the loaded working directory. It allows quick navigation and opening of documents via double-click. To maximize workspace, the panel can be hidden.
- **Tab Manager:** The central workspace allowing simultaneous editing of multiple files. Tabs visually indicate unsaved changes (with an asterisk) and protect the user against data loss upon closing the window.
- **Editor Control Panel:** Contains quick toggles (checkboxes) to show or hide the live preview and the analyzer output.
- **Live Preview:** A panel located to the right of the text editor that renders formatted HTML output (including code syntax, tables, and links) in real-time based on the typed text.
- **Analyzer Output:** A bottom panel within the tab that interactively lists the results of structural checks and link validation. It distinguishes successful checks, warnings, and critical errors using colors (red, yellow, or green).

5.3 Git Scene

This view serves for integration with the version control system and provides a graphical interface for repository management.

- **Commit Message Input Field:** A multi-line text box that enforces the entry of a commit message before changes can be pushed.
- **Git Control Panel:** Buttons for launching asynchronous Git operations (Status, Fetch, Pull, Push, Export Staged).
- **Output Console:** A text output positioned in the center, providing the user with direct feedback on running Git operations and logging background auto-staging.

6 Testing

This section describes the process and methodology for verifying the correct functionality of the application.

6.1 Unit Tests

The desktop application's unit tests are written in the `pytest` framework and are located in the `application/tests/` directory. The tests are thematically divided into the subdirectories

`analyzers/`, `parsers/`, `file/`, and `git/`, which mirror the structure of the production code. Shared fixtures (e.g., initializing an empty Git repository with an initial commit) are defined in `conftest.py` and `tests/fixtures/`. Tests are executed from the `application/` directory using the command `pytest tests/` (optionally with `-v` for verbose output or `--cov=core --cov=utils` for coverage measurement). Each test is independent – it utilizes the `tmp_path` pytest fixture for temporary files, so tests do not interfere with the actual file system or the project’s Git repository.

Analyzer Tests (`tests/analyzers/`)

- **`test_markdown_analyzer.py`** – Covers the `MarkdownAnalyzer` orchestrator: initialization with custom and default `base_path`, checks for H1 count, H1 filename match, H1 prefix, required sections, section order, unknown section detection, heading levels, breadcrumb and link validation, markdown link extraction, and text report generation.
- **`test_content_validator.py`** – Covers section content checks: empty mandatory sections, `found_types` matching `expected_types`, table columns and existence, mandatory bullet prefixes, detection of italic instructions carried over from the template, and A–Z block completeness checks.
- **`test_formatting_validator.py`** – Covers low-level formatting checks: unclosed bold and italic, images without alt text, table correctness, and Unicode *replacement* character detection.
- **`test_links_validator.py`** and **`test_relative_checking.py`** – Cover the core class `LinkValidator` from the core module: checking the existence of relative link targets, reciprocal links between patterns in the *Related Patterns* section, and alias consistency with the *Also Known As* section.

Parser Tests (`tests/parsers/`)

- **`test_base_parser.py`** – Covers shared parsing primitives: `parse_heading_line()` (heading levels, optional and variable markers, links in headings), `split_h1()`, `parse_breadcrumbs()` (distinguishing from italic instructions), and `classify_content_line()` for all content types (text, bullet, table, image, link, footnote, horizontal rule).
- **`test_markdown_parser.py`** – Covers `MarkdownParser`: constructing a `ParsedDocument` from various inputs, correctly populating `DocumentMeta` (breadcrumbs, H1 prefix and value), collapsing A–Z blocks into group headings, and extracting aliases and cross-references via the `get_link_metadata()` method.
- **`test_template_parser.py`** – Covers `TemplateParser`: recognizing mandatory and optional sections, italic *variable* placeholders, content rules (`ContentRules`), correctly constructing `HeadingRules`, and collapsing A–Z blocks within the template.

File-layer tests (`tests/file/`)

- **`test_paths.py`** – Validates `Config` repository-root resolution (locating the `.git` directory in the working directory or parents, failing when none exists), default markdown paths, and `MarkdownAnalyzer` initialization with respect to `base_path`.

- **test_file_helper.py** – Covers `FileHelper`: UTF-8 and CP1252 read/write round-trips, rejecting saves that violate a CP1252-locked file, `find_markdown_files` (recursive and flat layouts, deduplication), `get_file_info`, `resolve_relative_markdown_link`, validation helpers, `MAX_FILE_SIZE` MB limits, and error paths (`FileNotFoundError`, `FileSizeError`, `FileWriteError`).
- **test_file_matcher.py** – Covers `FileMatcher`: template assignment from parent/grand-parent directory segments (`catalogue/`, `publications/categories/methodologies`, etc.), facets index files wired to `facets_index_template`, unknown paths yielding no match, and backslash-separated paths on Windows-like input.

Git-operation tests (`tests/git/`)

- **test_git_runner.py** – Targets helpers in `utils/git/runner.py`: path normalization relative to repo root, markdown-extension detection, finding the repo root from a subdirectory, Markdown-only status summaries, staged/unstaged wording in formatted status messages, and clean-vs-dirty trees.
- **test_git_operations.py** – Covers the refactored `utils/git/` helpers plus `GitManager`: `stage_markdown_files` filtering, `stage_file` success and rejection of out-of-repo paths, `unstage_all`, `commit` (empty or over-long messages, missing staged changes, `stage_all`), `push_markdown_changes` (custom vs default vs empty-supplied-message behavior, rejecting over-long messages, blocking when mocked sync indicates the branch is behind the remote), `fetch/pull/push` guards when remotes/sync state are unsafe, `export_staged_files.z`, `GitOperationWorker` routing for unknown and known ops, `GitManager` rejecting overlapping operations while busy, and export flows that depend on the confirmation dialog outcome.

6.2 Test Scenarios

To comprehensively verify the functionality of the application from the user’s perspective, the correct behavior of the user interface, error handling, and Git version control system integration, a set of manual test scenarios was created. These scenarios cover standard application workflows (happy paths) as well as non-standard behavior and edge cases. A complete list of all test scenarios can be found in **Appendix A.2**.

7 Future Extensions

This section contains a list of further possible extensions to the application.

- **File Structure Outline** – Display the file’s structure as a hierarchy of sections. The user should have the option to choose whether this hierarchy is displayed. Clicking on a section should move the editor to that specific section in the markdown file.
- **Multilingual Support** – Support for multiple languages. Currently, it primarily supports English.
- **Export to .dot Format** – Exporting the file in `.dot` format, which would display a graph structure of connections based on the links within the markdown files.

- **Creating New Files** – By default, the application only modifies the content of existing markdown files from the hard drive. In the future, it could include the ability to create new markdown files.
- **Moving/Renaming Files and Directories** – By default, the application only opens existing files from the hard drive. In the future, it could include the ability to move and rename files and directories.
- **Improved Git Commit** – Splitting the lines of markdown files into multiple commits.

8 Build Guide

This section summarizes the installation, build, and execution of the application. More detailed English guides are available directly in the repository, complemented specifically by these files:

- **application/README.md** – User manual (UI controls, basic tips for analysis, and Git mode).
- **application/INSTALLATION_OVERVIEW.md** – A hub for developer and user installation instructions.

The concept differentiates between a *regular user* and a *developer*: an end-user typically runs an already built program or obtains an executable file from a provided script; a developer works with the source code, a Python virtual environment, and dependencies located in `application/requirements.txt`. It is recommended to run the application from the project's root folder (containing the project catalog and Git metadata); running from the wrong working directory may cause the program to launch improperly.

8.1 Requirements

- **Git** – If Git integration features are used, Git must be installed on the system (git-scm.com/downloads). Verification: use the `git --version` command.
- **Python 3.10** – Required for development and manual dependency installation; see below. Installation documentation: python.org/downloads.

8.2 End User: Running the Program

If a pre-built executable file is available, there is no need to manually create a virtual environment or install dependencies from PyPI.

Windows – in the application directory or within the `dist` folder (PowerShell also accepts forward slashes):

```
./markdown_analyzer.exe
./dist/markdown_analyzer.exe
```

Linux / macOS – in the application directory or within the `dist` folder:

```
./markdown_analyzer
./dist/markdown_analyzer
chmod +x ./dist/markdown_analyzer # if the system denies execution
```

8.3 Building the Executable (Build Scripts)

Build scripts are located in the **root of the cloned repository** (next to the `application/` folder). Once the build is complete, the output is typically found in the `dist/` folder.

Windows (PowerShell) – Path notation with forward slashes is equivalent to the standard form with backslashes:

```
cd C:/path/to/project_root
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
./build_script.ps1
./dist/markdown_analyzer.exe
```

Linux / macOS:

```
cd /path/to/project_root
chmod +x build_script.sh # once, if the script lacks execution rights
./build_script.sh
./dist/markdown_analyzer
```

8.4 Developer Environment and Dependencies

Working with the source code recommends using a Python virtual environment and installing packages from `application/requirements.txt`.

1. Verify the interpreter version (Python 3.10.x), e.g., `python3 --version` or `py --version` on Windows.
2. Clone the repository and navigate to the application folder:

```
git clone <HTTPS/SSH>
cd .../application
```

3. Create and activate the virtual environment:

```
python3.10 -m venv venv # Linux / macOS
py -3.10 -m venv venv # Windows, if python3.10 is missing
source venv/bin/activate # Linux / macOS
# Windows (PowerShell): ./venv/Scripts/Activate.ps1
```

4. Install dependencies:

```
pip install -r requirements.txt
# alternative: python -m pip install -r requirements.txt
```

5. Build the package as described in the previous subsection from the **root** of the repository (`build_script.ps1` or `build_script.sh`), or run the module directly during debugging from the activated virtual environment inside `application`, e.g., `python main.py`.

8.5 Notes

- Python versions older than 3.10 may cause dependency issues.
- On Windows, the default PowerShell policy may block script execution; a solution is to temporarily bypass it for the process (see above) or configure it according to the system documentation.

- If Git features fail, check the Git installation on the system.

A Appendices

A.1 Diagrams

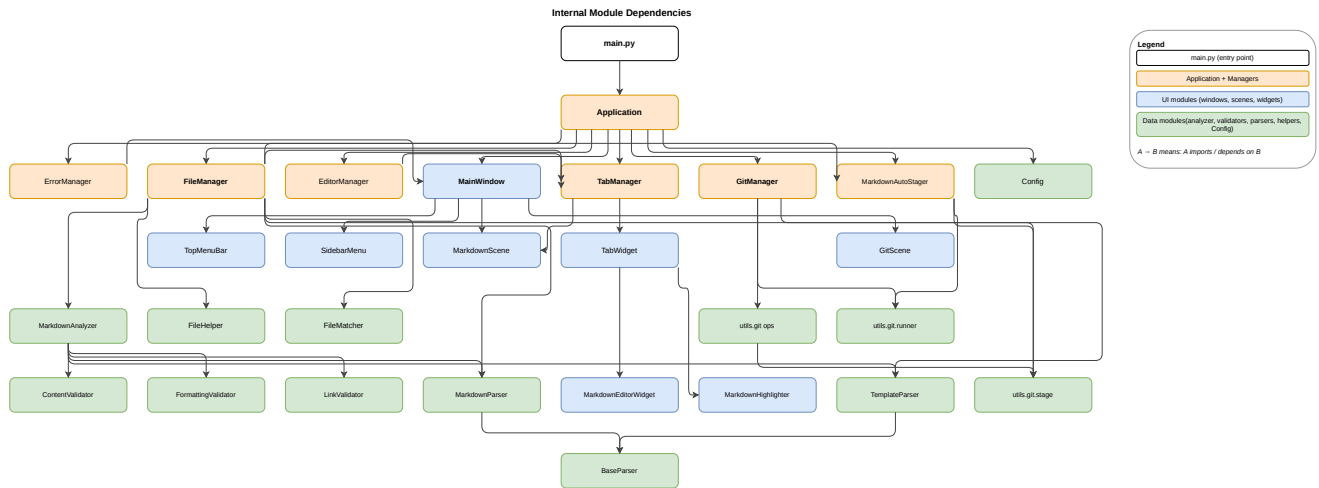


Figure 1: Dependencies of the main application modules (imports and layers between the entry point, application, managers, user interface, and helper libraries).

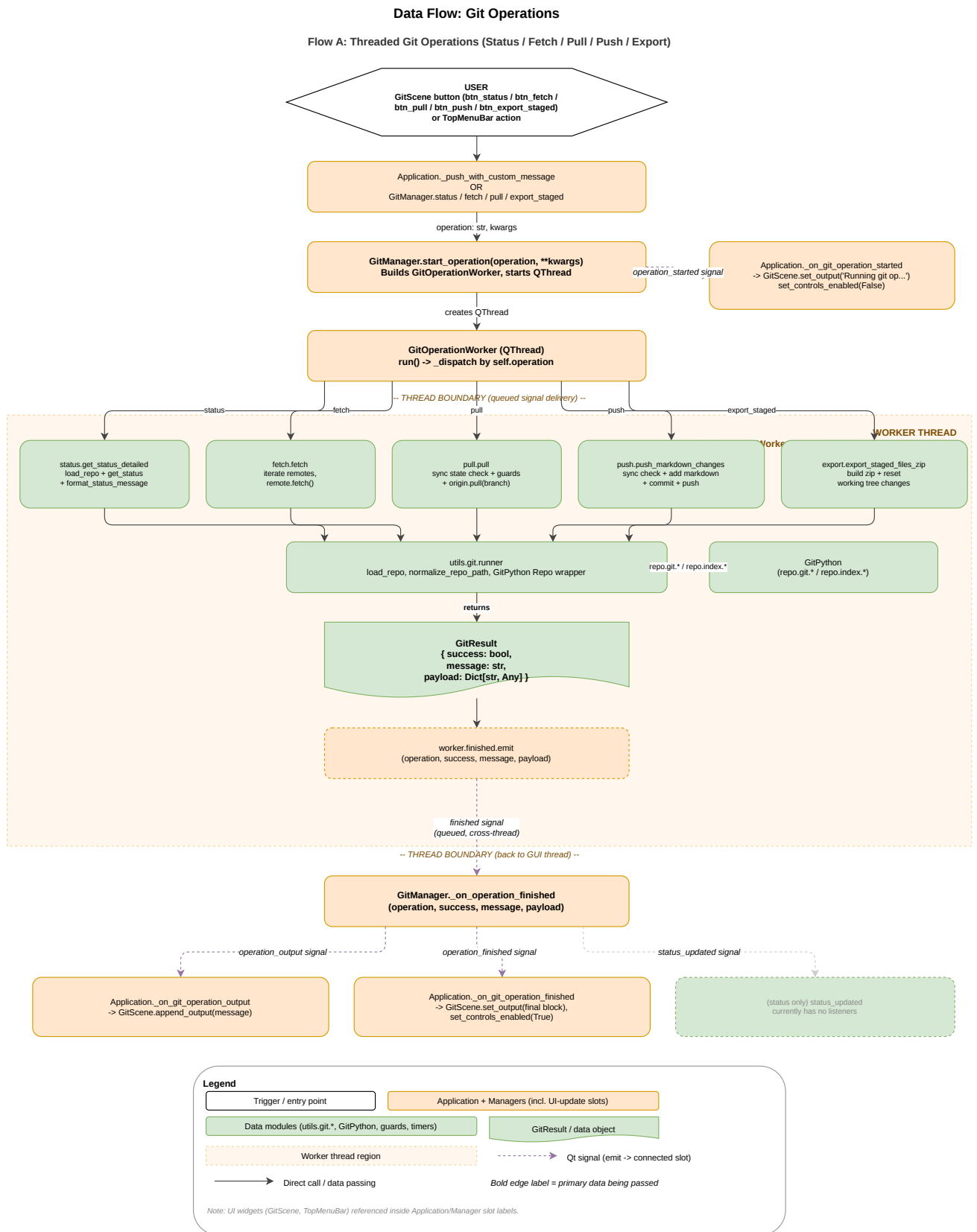


Figure 2: Data flow during Git operations (Flow A): user action in Git Scene or menu, orchestration in GitManager, execution of commands in the GitOperationWorker thread using the utils/git package and GitPython, and feedback to the interface console.

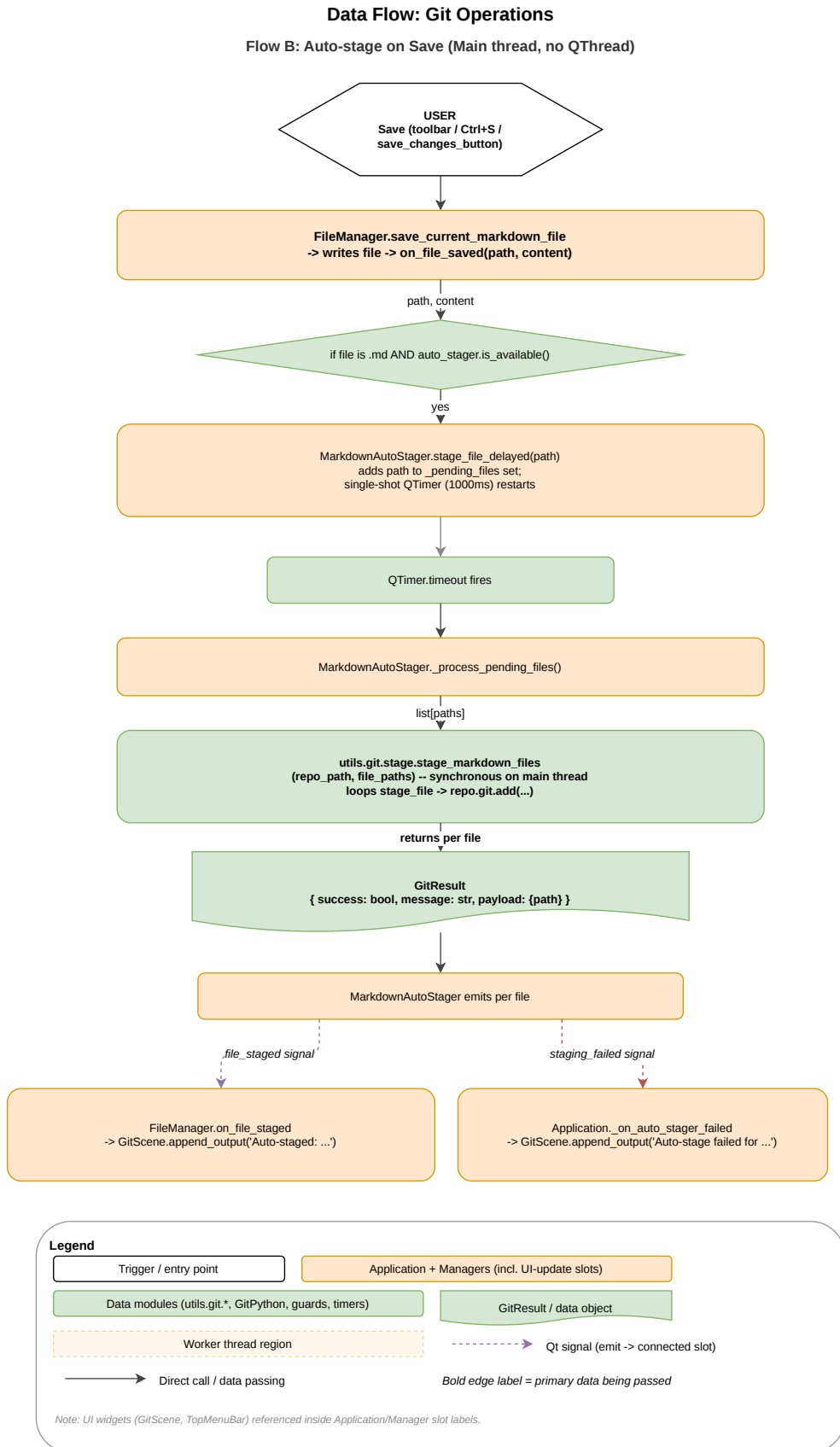


Figure 3: Data flow during Git operations (Flow B): background automatic change staging (Auto-stager) triggered after saving a file.

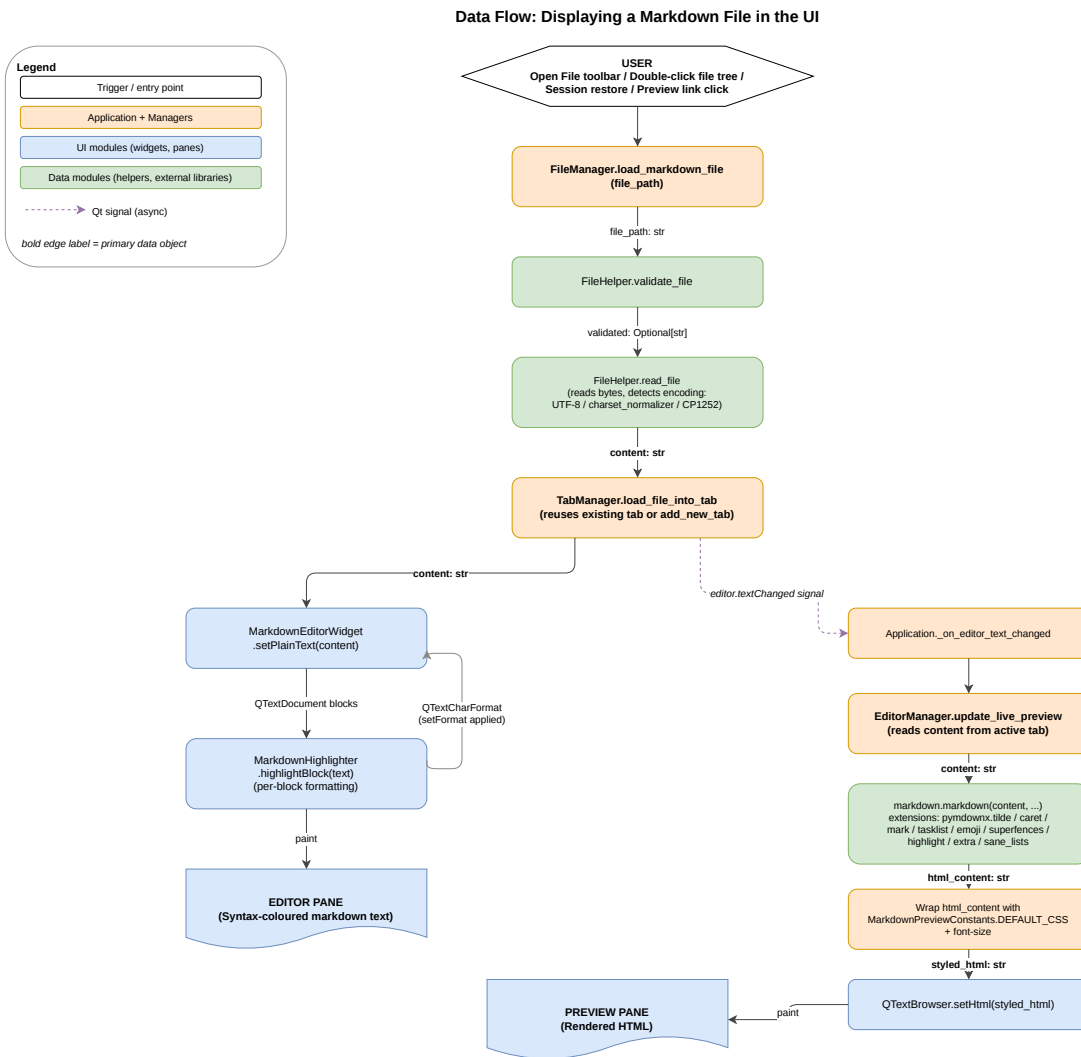


Figure 4: Data flow during markdown file display: **FileManager** uses **FileHelper** for content validation and loading, **TabManager** handles tabs and the editor; the live preview is generated by converting markdown to HTML and rendering it in the preview widget.

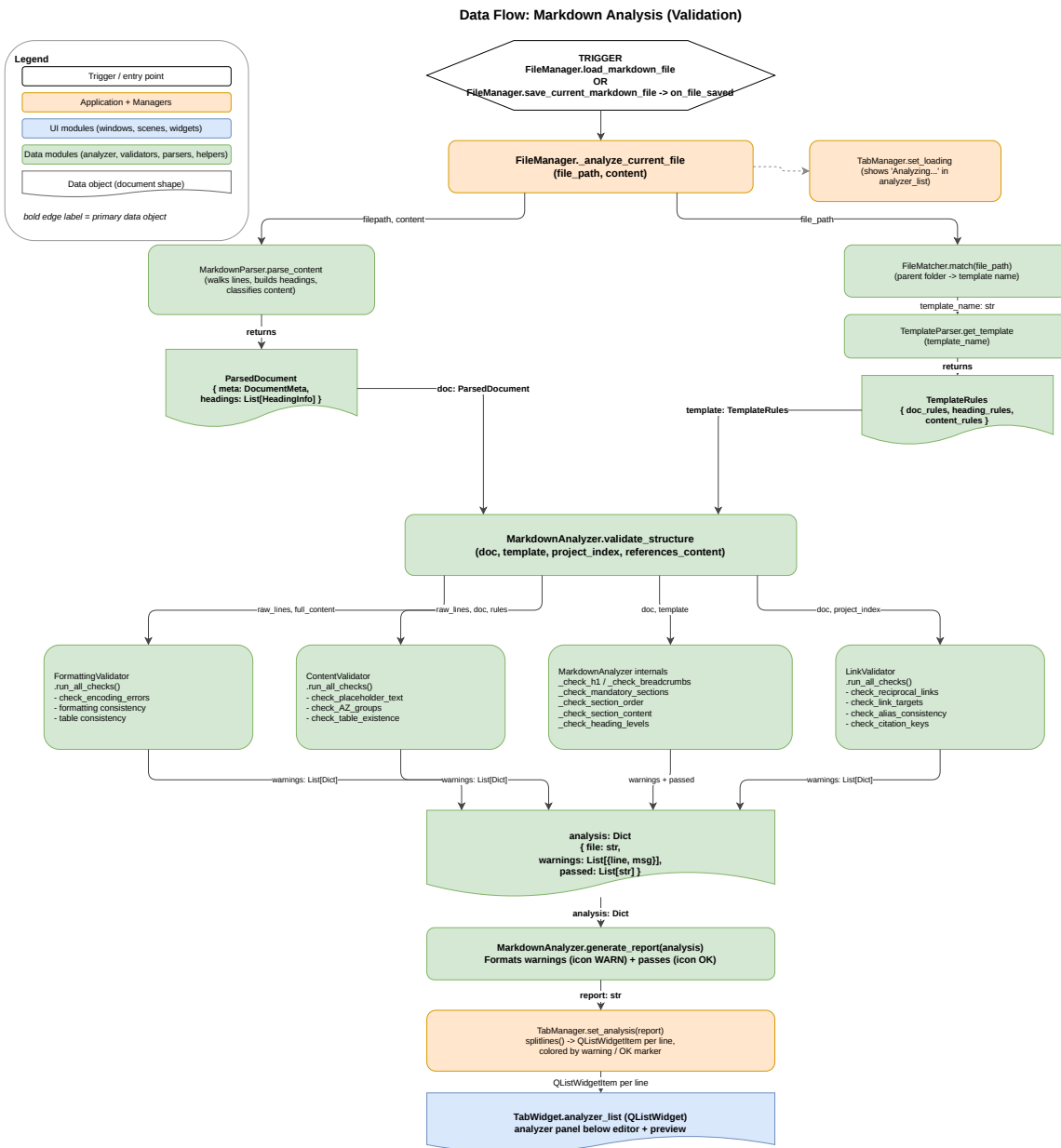


Figure 5: Data flow during markdown structure analysis and validation upon loading or saving: from `FileManager` through document parsing, template selection, down to the validators and printing of results in the analyzer panel.

A.2 Test Scenarios

This document contains the main functional tests for the application, focusing on the user interface, error handling, and Git integration.

1. Git Operations

Scenario 1.1: Execute Git Push with an entered commit message

Description: Verify that the application executes a push when the user enters a commit message.

- **GIVEN** the user is in the Git view (Git scene) and has staged changes.
- **AND** the text field for the commit message contains text.
- **WHEN** the user clicks the "Push" button.
- **THEN** the background Git push operation EXECUTES.

Scenario 1.2: Prevent Git Push with an empty commit message

Description: Verify that the application blocks a push if the user forgets to enter a commit message.

- **GIVEN** the user is in the Git view (Git scene) and has staged changes.
- **AND** the text field for the commit message is completely empty (or contains only spaces).
- **WHEN** the user clicks the "Push" button.
- **THEN** a critical dialog appears with the title "Missing Commit Message".
- **AND** the background Git push operation DOES NOT EXECUTE.

Scenario 1.3: Export changed files

Description: Verify that the user successfully exports changed files from the working directory.

- **GIVEN** the user is in the Git view.
- **WHEN** the user clicks the "Export Staged" button.
- **THEN** a warning dialog appears with the title "Warning: Potential Data Loss".
- **WHEN** the user clicks "Yes".
- **THEN** the dialog closes and the export operation executes.

Scenario 1.4: Prevent unintended data loss during Git Export

Description: Verify that the user is warned prior to exporting, as this action clears the working directory.

- **GIVEN** the user is in the Git view.
- **WHEN** the user clicks the "Export Staged" button.
- **THEN** a warning dialog appears with the title "Warning: Potential Data Loss".
- **WHEN** the user clicks "No".
- **THEN** the dialog closes and the export operation is canceled.

2. Application Exit and Tab Management

Scenario 2.1: Closing the application with unsaved tabs (Cancel)

Description: Verify that the user can abort closing the application if there is unsaved work.

- **GIVEN** the application is open and contains at least one unsaved tab (indicated by an asterisk * in the tab name).
- **WHEN** the user clicks the system button to close the window ('X').
- **THEN** a warning dialog appears prompting to Save, Discard, or Cancel.
- **WHEN** the user clicks "Cancel".
- **THEN** the dialog closes and the application remains open.

Scenario 2.2: Saving unsaved tabs when closing the application

Description: Verify that the user can successfully save their work during the application exit process.

- **GIVEN** the application is open with an unsaved tab.
- **WHEN** the user clicks the system button to close the window ('X').
- **AND** a warning dialog appears prompting to Save, Discard, or Cancel.
- **WHEN** the user clicks "Save".
- **THEN** the given file is saved.
- **AND** the application terminates.

Scenario 2.3: Discarding changes when closing the application

Description: Verify that the user can intentionally discard changes and force the application to close.

- **GIVEN** the application is open with an unsaved tab.
- **WHEN** the user clicks the system button to close the window ('X').
- **AND** a warning dialog appears prompting to Save, Discard, or Cancel.
- **WHEN** the user clicks "Discard".
- **THEN** the unsaved changes are ignored and the application terminates.

3. File Explorer and Link Validation

Scenario 3.1: Safely handling invalid relative links

Description: Verify that clicking a broken link in the live preview does not crash the application.

- **GIVEN** the user is viewing a markdown file in the editor.
- **AND** the live preview panel is visible.
- **WHEN** the user clicks on a relative link in the preview that points to a non-existent file (e.g., ./missing_document.md).
- **THEN** a critical dialog appears with the title "Link Error".

- **AND** the application remains stable and active.

Scenario 3.2: Safely handling external file deletion

Description: Verify that the application starts up safely even if a previously opened file was deleted by the operating system.

- **GIVEN** the user has the file `notes.md` open in a tab.
- **AND** the user closes the application.
- **WHEN** the user deletes the file `notes.md` using the file explorer in Windows/macOS.
- **AND** the user launches the application again.
- **THEN** the application starts successfully without crashing.
- **AND** the tab for `notes.md` either does not restore, or displays a clean, empty workspace without the opened tab.

4. Settings and State Persistence

Scenario 4.1: Remembering window dimensions and layout

Description: Verify that the application restores its exact size and screen position.

- **GIVEN** the application is running in windowed mode (not maximized).
- **WHEN** the user changes the window size to a specific dimension and moves it to a corner of the screen.
- **AND** the user closes and relaunches the application.
- **THEN** the application opens in the exact same position and size on the screen.

Scenario 4.2: Restoring previously opened tabs

Description: Verify that the user's workspace is preserved exactly as left.

- **GIVEN** the user has exactly three different markdown files open in three tabs (`A.md`, `B.md`, `C.md`).
- **AND** tab `B.md` is currently active/focused.
- **WHEN** the user closes the application (saving any changes).
- **AND** the user relaunches the application.
- **THEN** all three tabs (`A.md`, `B.md`, `C.md`) are automatically restored.
- **AND** tab `B.md` is set as active/focused.

Scenario 4.3: Restoring previously opened working directory

Description: Verify that the opened working directory in the Explorer panel is preserved exactly as left.

- **GIVEN** the user has a working directory open.
- **WHEN** the user closes the application.
- **AND** the user relaunches the application.
- **THEN** the given working directory is displayed in the Explorer panel.

5. Live Editor and Interface Toggles

Scenario 5.1: Rendering the live preview in real-time

Description: Verify that the live preview reacts instantly to changes in the editor.

- **GIVEN** the user has an open markdown file.
- **AND** the "Live Preview" checkbox is checked and the preview panel is visible.
- **WHEN** the user types *# New heading* into the text editor.
- **THEN** the live preview panel updates immediately and displays the formatted *H1* HTML heading without needing to save the file.

Scenario 5.2: Toggling UI panels (specifically "Analyzer")

Description: Verify that the UI cleanly hides and restores panels.

- **GIVEN** the user is in the main view with markdown.
- **WHEN** the user unchecks the "Analyzer" checkbox.
- **THEN** the analyzer list completely disappears from the layout, freeing up more screen space for the editor.
- **WHEN** the user checks the "Analyzer" box again.
- **THEN** the analyzer list reappears and retains its previous content.

Scenario 5.3: Toggling UI panels (specifically "Live Preview")

Description: Verify that the UI cleanly hides and restores panels.

- **GIVEN** the user is in the main view with markdown.
- **WHEN** the user unchecks the "Live Preview" checkbox.
- **THEN** the live preview completely disappears from the layout, freeing up more screen space for the editor.
- **WHEN** the user checks the "Live Preview" box again.
- **THEN** the live preview reappears with updated visuals if the content in the markdown file changed.

6. Markdown Analyzer Core

Scenario 6.1: Flagging missing required headings

Description: Verify that the analyzer detects template structure violations.

- **GIVEN** the user has loaded a project directory containing a template (`template_pattern.md`) that requires the heading *## Tasks*.
- **WHEN** the user opens a file matching this template category, but the heading *## Tasks* is missing from the file.
- **THEN** a specific warning/error appears in the Analyzer panel notifying of the missing required structure.

Scenario 6.2: Analyzer clears errors after correction

Description: Verify that the analyzer updates dynamically when the user fixes an error.

- **GIVEN** the Analyzer panel currently displays a structural error (e.g., missing mandatory heading).
- **WHEN** the user adds the missing heading into the editor.
- **AND** the user clicks "Save Changes".
- **THEN** the error is removed from the Analyzer panel and a "Passed" or "OK" status is displayed.

7. File Explorer Integration

Scenario 7.1: Double-click opens a new tab

Description: Verify that the explorer correctly opens new tabs.

- **GIVEN** the user has a project folder open in the Explorer panel.
- **AND** the file `sample_pattern.md` is NOT currently open in any tab.
- **WHEN** the user double-clicks `sample_pattern.md` in the Explorer tree.
- **THEN** a new tab is created for `sample_pattern.md`.
- **AND** the new tab becomes active (focused).

Scenario 7.2: Double-click on an already open file focuses it

Description: Verify that the explorer does not create duplicate tabs for the same file.

- **GIVEN** the user has a project folder open in the Explorer panel.
- **AND** the file `sample_pattern.md` is already open in Tab 1 (the tab has the same name as the file).
- **AND** the user is currently viewing Tab 2.
- **WHEN** the user double-clicks `sample_pattern.md` in the Explorer tree.
- **THEN** the application DOES NOT CREATE a duplicate tab.
- **AND** the application immediately switches focus back to Tab 1.

8. Background Auto-Stager

Scenario 8.1: Saving a file triggers the auto-stager

Description: Verify that the Git manager silently stages files in the background.

- **GIVEN** the user has a tracked markdown file open in the editor.
- **AND** the Git repository has no uncommitted changes.
- **WHEN** the user makes a text edit and clicks "Save Changes".
- **THEN** the file is saved to disk.
- **AND** an output message is appended to the Git view console confirming the specific file was successfully staged (auto-staged) in the background.

9. Top Menu & Shortcuts

Scenario 9.1: Opening files and folders via the File menu

Description: Verify that menu items and opening shortcuts trigger the correct system dialogs.

- **GIVEN** the application is running and active.
- **WHEN** the user selects *File* → *Open File* in the top menu (or presses *Ctrl+O*).
- **THEN** a system dialog opens for selecting an *.md* file.
- **WHEN** the user selects *File* → *Open Folder* (or presses *Ctrl+F*).
- **THEN** a system dialog opens for selecting a project directory.

Scenario 9.2: Saving via keyboard shortcut

Description: Verify that the save keyboard shortcut correctly saves the active tab.

- **GIVEN** the user has an active tab with unsaved changes (indicated by *).
- **WHEN** the user selects *File* → *Save File* (or presses *Ctrl+S*).
- **THEN** the current file is saved to disk.
- **AND** the unsaved changes indicator disappears immediately.

Scenario 9.3: Toggling panels via the File menu (Toggles)

Description: Verify that the menu correctly synchronizes the state of UI panels.

- **GIVEN** the user is in the main view (Markdown scene).
- **WHEN** the user selects *File* → *Open Explorer* (or presses *Ctrl+E*).
- **THEN** the file explorer sidebar hides (if it was visible) or shows (if it was hidden).
- **AND WHEN** the user presses *Ctrl+P* (Live Preview) or *Ctrl+A* (Analyzer).
- **THEN** the respective panels in the editor show or hide and their checkboxes update automatically.

Scenario 9.4: Launching basic Git operations via the menu

Description: Verify that keyboard shortcuts trigger background processes correctly.

- **GIVEN** the user has a repository open.
- **WHEN** the user selects *Git* → *Status* (or presses *Ctrl+Shift+S*).
- **THEN** the operation starts.
- **AND** even if the user is not currently in the Git view (Git scene), they will see the result in the console upon switching to it.
- *(Note: This test repeats analogously for Fetch / Ctrl+Shift+F and Pull / Ctrl+Shift+L).*

Scenario 9.5: Safeguards for critical Git actions via the menu

Description: Verify that triggering risky actions via menu or shortcut does not bypass our safeguards (message validation and data loss warnings).

- **GIVEN** the commit message text field is empty.
- **WHEN** the user presses *Ctrl+Shift+P* (Push).

- **THEN** the operation stops and a "Missing Commit Message" error dialog pops up.
- **AND WHEN** the user presses *Ctrl+Shift+E* (Export Staged).
- **THEN** a "Warning: Potential Data Loss" dialog pops up just as it would if the user clicked the physical button.